

MDI3003

ADVANCED PREDICTIVE ANALYTICS

**Constructing a Recommendation System from Customer
Transaction Data Using Random Forest**

NAME : Harshita Bogineni

REG No :23MID0043

COURSE CODE : MDI3003

COURSE TITLE : Advanced Predictive Analytics

FACULTY DETAILS : Dr.Durgesh Kumar

GithubLink:https://github.com/Harshita2011/Advanced_Predictive_Analytics_LAB

Contents

1. Aim and Learning Outcomes
 2. Executive Summary
 3. Problem Formulation and Predictive Translation
 4. Dataset Description, Provenance and Integrity
 5. Methodology and Experimental Protocol
 6. Candidate Generation and Negative Sampling
 7. Feature Engineering
 8. Model Development
 9. Exploratory Data Analysis and Visualization
 10. Evaluation Framework
 11. Evaluation Results
 12. Cross-Dataset Synthesis
 13. Interpretation
 14. Error, Cold-Start and Bias Analysis
 15. Responsible Recommendation and Privacy
 16. Limitations and Risks
 17. Future Improvements
 18. Reproducibility and Artifacts
 19. QP Compliance Matrix
 20. Conclusion
- References

1. Aim and Learning Outcomes

Aim: To construct, evaluate, and interpret a personalized recommendation system from customer transaction data using Random Forest as the primary supervised ranking/scoring model, evaluated against popularity, item–item collaborative filtering, and matrix-factorization baselines under a leakage-safe chronological protocol.

Learning outcomes demonstrated: (i) transformation of transaction logs into leakage-safe user–item learning examples; (ii) construction of popularity and heuristic baselines; (iii) engineering of customer, item, interaction, RFM and affinity features; (iv) training and tuning of a Random Forest classifier using only past information; (v) generation of personalized Top-K ranked recommendations; (vi) evaluation with both pair-classification diagnostics and recommendation-ranking metrics; (vii) analysis of cold-start, popularity bias, sparsity and failure cases; (viii) comparison against recommendation-native baselines with bootstrap uncertainty; and (ix) documentation of provenance, privacy boundaries and reproducibility artifacts.

2. Executive Summary

This report presents the design, implementation, evaluation and interpretation of a personalized recommendation system built from customer transaction data, in which a Random Forest classifier serves as a supervised purchase-propensity ranker. Three datasets are evaluated: Dataset 1 – Initial Retail Benchmark (a synthetic 18,765-interaction benchmark used for pipeline validation), Dataset 2 – UCI Online Retail (QP D1; 541,909 raw transactions, 392,692 cleaned, 4,338 customers, 3,665 items) as the core dataset, and Dataset 3 – Retailrocket (QP D2; 1,407,580 clickstream events) as the advanced replication benchmark.

The central research question — whether a supervised Random Forest ranking pipeline can outperform a non-personalized popularity baseline on future-item recommendation while remaining reproducible, leakage-safe and computationally practical — is answered affirmatively on the core dataset, with important nuance. On the locked test window of 1,557 customers, Random Forest achieves Precision@5 = 0.0459 versus 0.0313 for Popularity (+47%), and HitRate@10 = 30.0% versus 20.9%; user-level bootstrap 95% confidence intervals for Random Forest and Popularity do not overlap on Recall@10 or NDCG@10, supporting a genuine ranking improvement. The Random Forest also nearly doubles catalog coverage relative to Popularity (3.61% versus 1.86% of the 3,542-item active catalog).

Equally important, the benchmarks delineate the method's boundary conditions. Item–Item CF and SVD dominate coverage (21.96% and 14.37%) and achieve the strongest NDCG@10 (0.0561 and 0.0732), so Random Forest is not the best ranking model universally; on the 99.98%-sparse Retailrocket clickstream, the Popularity baseline retains the advantage because feature-based scoring cannot manufacture signal from near-empty histories. Feature ablation confirms that item-popularity features carry the largest share of ranking signal, and negative-sampling sensitivity analysis shows the pipeline is robust across 1:20, 1:50 and 1:100 ratios. All results were produced under strict chronological splitting, a fixed random seed (42), and a final Random Forest configuration of `n_estimators = 200`, `max_depth = 12`, `min_samples_leaf = 2`, `max_features = 'sqrt'`.

3. Problem Formulation and Predictive Translation

Recommendation is formulated as a ranking problem over customer–item candidates rather than merely a binary classification task. Because Random Forest is not a native collaborative-filtering algorithm, it is employed as a supervised candidate-scoring model: learning examples are constructed from transaction history as customer–item pairs, the classifier estimates the probability of purchase within a future evaluation window, candidates are ranked by this score, and the Top-K items are returned.

Element	Operational definition
Recommendation decision	Rank eligible products for a customer using only information available before the recommendation moment t .
Target	Future purchase/interaction of the customer–item pair in a clearly defined evaluation window after t .
Positive pair	A customer–item pair purchased/interacted with in the future evaluation window ($y = 1$).
Negative pair	An eligible candidate item not purchased in the future window; sampled reproducibly from the candidate universe ($y = 0$).
Success criterion	Relevant future items appear in the Top-K list more often than under the non-personalized Popularity baseline, without collapsing catalog coverage.

Table 1. Predictive translation of the business problem into a supervised ranking task (QP §3.1).

3.1 Business Context and Stakeholders

In e-commerce, user–item interaction matrices are extremely sparse (99.4% on Dataset 2; 99.98% on Dataset 3). Recommending purely global bestsellers introduces popularity bias and catalog starvation, in which long-tail products remain permanently undiscovered. Customers suffer choice overload; merchandisers see distorted inventory and campaign decisions; the platform absorbs the asymmetric revenue cost of ranking errors at the top positions. A transparent Popularity baseline is retained throughout as the interpretable floor that any personalized model must beat.

4. Dataset Description, Provenance and Integrity

Dataset Naming and QP Mapping Note: Dataset 1 in this report refers to the UCI Online Retail data downloaded from the first dataset link provided under the QP's *References and Student-Help Links*. Dataset 2 corresponds to **D1 – UCI Online Retail** in the QP and contains **541,909 raw transactions**. Dataset 3 corresponds to **D2 – Retailrocket Recommender System Dataset**.

Report Label	QP Label	Dataset
Dataset 1	Student-help link dataset	UCI Online Retail
Dataset 2	D1	UCI Online Retail — 541,909 raw transactions
Dataset 3	D2	Retailrocket Recommender System Dataset

4.1 QP Dataset Mapping

Report dataset	Dataset name	QP mapping	Role in this study
Dataset 1	UCI	Not a QP dataset; instructor-style synthetic benchmark	Pipeline validation on a controlled, dense interaction regime
Dataset 2	UCI Online Retail(kaggle)	QP D1 – UCI Online Retail	Core dataset for all primary experiments (E1–E10 evidence)
Dataset 3	Retailrocket E-commerce	QP D2 – Retailrocket Recommender System Dataset	Advanced cross-dataset replication (E9) and extreme-sparsity stress test

Table 2. Explicit mapping of report datasets to the QP dataset catalogue (QP §5).

4.2 Dataset Summary

Dataset	Rows / events	Entities	Interaction modality	Task	Source and time span
Dataset 1 – Initial Retail Benchmark	18,765 interactions	1,000 users / 500 items	Synthetic invoices	Binary purchase (0/1) Top-K	UCI ML
Dataset 2 – UCI Online Retail (QP D1)	541,909 raw / 392,692 cleaned	4,338 customers / 3,665 StockCodes / 18,532 invoices	Real B2B/B2C transactions (Quantity, UnitPrice)	Next-period item purchase in Top-K	UCI ML Repository (ID 352); 01-Dec-2010 to 09-Dec-2011
Dataset 3 – Retailrocket (QP D2)	1,407,580 events	1.4M visitors / 235,000+ items	Clickstream funnel (view, add-to-cart, transaction)	Next-period transaction/cart item retrieval	Kaggle (Retailrocket); May 2015 – Sep 2015

Table 3. Summary of the three evaluated datasets.

4.3 Dataset Card — UCI Online Retail (Core, QP D1)

Attribute	Value
Source URL	https://archive.ics.uci.edu/dataset/352/online+retail
Access date	Fall Semester 2026–2027 study period
Version / checksum	SHA-256: f0bbca3b43db045c47936a5c1e550e5ebc1cce8a370e0f80bb8b8159bca4ca26
Raw transaction rows	541,909 rows × 8 columns
Missing CustomerID removed	135,080 rows (24.93%)
Cancellations / invalid (Qty or Price ≤ 0) removed	8,945 rows (1.65%)
Exact duplicates removed	5,192 rows (0.96%)
Cleaned transaction rows	392,692 rows (72.46%)
Unique customers	4,338
Unique catalog products (StockCode)	3,665
Unique invoices	18,532
Date range	2010-12-01 08:26:00 to 2011-12-09 12:50:00
Return/cancellation policy	Invoice numbers prefixed with "C" removed; Quantity > 0 and UnitPrice ≥ 0 enforced; returned items never counted as positive recommendations
Missing-ID policy	Rows without a usable CustomerID dropped from personalized modelling; exact count (135,080) recorded above
Privacy/usage note	Customer IDs used for grouping and joins only, never as ordinal numeric features; no direct identifiers exposed in outputs

Table 4. Dataset card for the core UCI Online Retail extract, exactly as recorded in the preprocessing audit.

4.4 Transaction Integrity Rules

Four integrity rules are enforced throughout: (i) cancellations and returns are removed explicitly rather than silently counted as positive relevance; (ii) exact duplicate lines are deduplicated (5,192 rows) while legitimate repeated quantities within an order are preserved; (iii) every feature aggregate is recomputed from the history available at each cutoff, never from validation/test transactions; (iv) customer identifiers serve as join keys only — all model inputs are history-derived features, never raw IDs.

5. Methodology and Experimental Protocol

5.1 End-to-End Workflow

A sixteen-step workflow is implemented: (1) define the recommendation moment and future target window; (2) load and audit transactions; (3) clean cancellations/returns and unusable customer IDs; (4) create chronological train/validation/test windows; (5) construct historical customer and item statistics from training history only; (6) build the Popularity baseline; (7) generate candidate items per customer; (8) create positive and sampled-negative user–item pairs; (9) build leakage-safe features; (10) train and tune the Random Forest on training/validation data only; (11) score candidates for locked-test customers; (12) filter ineligible items per the documented policy; (13) rank by predicted probability and return Top-K; (14) evaluate ranking metrics against future purchases; (15) inspect feature importance, cold-start and failure cases; (16) save the model, feature schema, candidate policy, predictions and reproducibility manifest.

5.2 Chronological Splitting and Leakage Prevention

Strict chronological quantile cutoffs are applied to the core dataset, avoiding any random shuffling: t_1 (70th percentile) = 2011-10-07 13:17:00 and t_2 (85th percentile) = 2011-11-11 12:10:00. The locked test window deliberately covers the holiday-season period, making the evaluation conservative. Acceptance tests assert that the maximum training timestamp precedes t_1 , that train and test indices are disjoint, and that every feature aggregate is recomputed at each cutoff. Exact cutoff timestamps for Datasets 1 and 3 are recorded in their respective split manifests (outputs/.../split manifests) and the same percentile rule is applied.

Split window	Date range	Rows	Unique customers	Unique products
Training history (< t_1)	2010-12-01 08:26 to 2011-10-07 13:16	274,819	3,702	3,542
Validation target [t_1 , t_2)	2011-10-07 13:17 to 2011-11-11 12:07	58,924	1,618	2,709
Locked-test target ($\geq t_2$)	2011-11-11 12:10 to 2011-12-09 12:50	58,949	1,557	2,637

Table 5. Exact chronological split of UCI Online Retail (cutoffs t_1 = 2011-10-07 13:17, t_2 = 2011-11-11 12:10).

All features for validation and test examples are constructed exclusively from information available before the corresponding prediction cutoff. Global item popularity, RFM statistics and pair affinities are never computed from validation or test transactions. Random customer-level splits were deliberately avoided, since they allow future transactions to leak into training and produce unrealistically optimistic estimates.

5.3 Models Evaluated

Four systems are compared under identical frozen splits and candidate policies on the core dataset: (i) the Popularity baseline — a non-personalized Top-K over the globally most-purchased training-history items; (ii) Item–Item Collaborative Filtering — cosine co-purchase similarity to the customer's historical items; (iii) Matrix Factorization — TruncatedSVD with 20 latent factors on the implicit interaction matrix; and (iv) the Random Forest point-wise ranker, whose positive-class probability $P(y=1 \mid x)$ is used as the ranking score. Model selection relied on validation Recall@K / NDCG@K and never on locked-test results; ROC-AUC and PR-AUC are retained strictly as pair-classification diagnostics.

5.4 Final Hyperparameter Configuration

Parameter	Value
n_estimators	200 (final locked model; ablation/sensitivity fast-loops used n = 150 for computational efficiency)
max_depth	12
min_samples_leaf	2
max_features	sqrt
Class imbalance	balanced_subsample class weighting
Random seed	42 (all splits, sampling and models)
Model selection	Validation ranking performance (Recall@K / NDCG@K); PR-AUC secondary

Table 6. Final Random Forest configuration, standardized across source code, serialized models, execution logs and this report.

5.5 Experiment Design Matrix

Experiment	QP ref	Purpose	Core-dataset (D2) setting
E1 – Popularity baseline	QP §18	Non-personalized reference floor	Top-K over training-history purchase counts, already-purchased items excluded from ranking per policy
E2 – Random Forest scorer	QP §18	Supervised time-valid user–item ranking	18 leakage-safe features; 1:50 candidate-aware negative sampling; tuned on validation
E3 – K sensitivity	QP §18	Stability of conclusions across K	K = 5, 10, 20 for all four systems
E4 – Feature ablation	QP §18	Contribution of feature groups	Full 18 features vs removal of pair-history, customer-RFM and item-popularity groups
E5 – Negative-sampling sensitivity	QP §18	Robustness to negative ratio	Ratios 1:20, 1:50, 1:100 compared on development pairs
E6 – Item–Item CF	QP §18 / advanced	Interaction-native benchmark	Cosine similarity over $3,542 \times 3,542$ co-purchase matrix
E7 – Matrix factorization	QP §18 / advanced	Latent-factor benchmark	TruncatedSVD, 20 latent factors, implicit matrix
E8 – Candidate-recall audit	QP §11.3	Retrieval-stage ceiling before ranking	Validation 96.65%; locked test 94.87%; per-user representability reported

E9 – Cross-dataset replication	QP §18	Generalization across data regimes	Retailrocket clickstream (Dataset 3); Initial Retail Benchmark (Dataset 1)
E10 – Responsible recommendation	QP §23	Bias, coverage and privacy audit	Coverage concentration, five-case audit, data minimization, privacy boundary

Table 7. Experiment design matrix mapped to QP requirements.

6. Candidate Generation and Negative Sampling

6.1 Candidate Universe

Training on every possible customer–item pair is infeasible and would create an enormous, unrealistic negative class. The candidate universe for the core dataset comprises all 3,542 products with at least one buyer in the training history. Under the documented repeat-purchase policy, previously purchased items remain eligible for recommendation, since online retail is a repeat-purchase domain; this policy is recorded in the candidate-policy artifact. Datasets 1 and 3 use bounded pools (Top-300 and Top-1,500 respectively) derived from training-history frequency.

Although the QP suggests an approximate candidate range of 500–2,000 items, the full 3,542-item active training catalog was retained for the core UCI experiment because restricting it further would reduce candidate recall; the resulting 94.87% locked-test candidate recall provides an empirical justification for this choice

6.2 Candidate-Recall Audit

Candidate recall — the fraction of true future-positive items present in the evaluation candidate universe — is measured before any ranker evaluation, because it imposes a hard ceiling on achievable Recall@K. Unretrievable future positives are retained in the audit and never silently discarded.

Evaluation window	Total future positives	Positives in candidate universe	Candidate recall	Users 100% representable
Validation target [t1, t2)	53,085	51,304	96.65%	961 / 1,618 (59.39%)
Locked-test target ($\geq$ t2)	50,985	48,372	94.87%	693 / 1,557 (44.51%)

Table 8. Candidate-recall audit for UCI Online Retail (Candidate_Recall.csv). Over 94.8% of future test purchases are representable within the candidate catalog; 44.51% of test users have all future positives representable.

6.3 Negative Sampling

Negative pairs are sampled reproducibly (numpy default_rng, seed 42) from the candidate universe excluding the customer's positives. The policy is candidate-aware — negatives are drawn from the same bounded universe the ranker scores at serving time — which aligns the training and serving distributions and avoids the easy-negative bias of arbitrary non-purchased catalog items. The retained configuration uses 50 negatives per positive; the sensitivity analysis in Section 11.5 justifies this choice.

6.4 Validation versus Locked-Test Metric Distinction

Sampled-candidate validation metrics rank one positive against ~50 sampled negatives, whereas locked-test metrics rank across the full candidate universe (and the Popularity/CF/SVD baselines rank across the whole active catalog). These measure fundamentally different tasks and are reported separately throughout; candidate recall is always reported alongside ranking metrics so the retrieval ceiling is never misattributed to the ranker.

7. Feature Engineering

All features are leakage-safe aggregates recomputed at each historical cutoff from training history alone. Customer identifiers are never used as model inputs.

Level	Features	Leakage control
Customer	cust_txns, cust_items, cust_qty, cust_spend (RFM aggregates)	Aggregated from train_hist only; cancellations removed before aggregation
Customer	cust_recency_days	Computed against cutoff timestamp t
Item	item_txns, item_buyers, item_qty, item_avg_price	History-window popularity only; never recomputed on validation/test
Item	item_recency_days	Velocity/trend signal from history
Customer–Item	pair_purchases, pair_qty, pair_spend, pair_recency_days	Pair affinity from history only; pair recency is a dominant split-gain feature
Customer–Item	share_of_spend, repeat indicator	Policy documented in candidate policy artifact
Customer–Item	co-purchase frequency	Co-occurrence from training baskets only
Context	month/season, country	Derived strictly before cutoff

Table 9. Feature dictionary with leakage controls (18 features in the full set).

8. Model Development

8.1 Popularity Baseline

Items are ranked by global purchase count computed from the training history; the Top-K not already purchased by the customer are returned. It is intentionally simple and establishes the non-personalized floor every personalized model must beat.

8.2 Random Forest Formulation

For cutoff t , features $x(u,i,t)$ are computed from history $H < t$, and $y = 1$ indicates that customer u purchases item i within the future target window. The forest aggregates 200 decision trees fitted on class-balanced bootstrap samples with random feature subsets; the positive-class probability is the ranking score: $\text{score}(u,i) = \text{RF.predict_proba}(\text{features}(u,i))[1]$; $\text{recommend}(u,K) = \text{top-K candidates ranked by score}$.

8.3 Advanced Benchmarks

Item–Item CF (cosine co-purchase similarity) and TruncatedSVD (20 latent factors) provide recommendation-native benchmarks, satisfying the QP requirement of comparing Random Forest against at least one collaborative-filtering or matrix-factorization method under the same chronological split, target definition and candidate policy. This frames the central comparison: does the Random Forest gain over Popularity justify its feature-engineering cost, and does a latent model improve ranking enough to justify its own complexity?

9. Exploratory Data Analysis and Visualization

All exploratory analysis uses training-period information only. The figures below cover the ten visualization requirements of QP §17; each figure is followed by an interpretation of what it means for recommendation quality or risk.

9.1 Dataset 1 – Initial Retail Benchmark

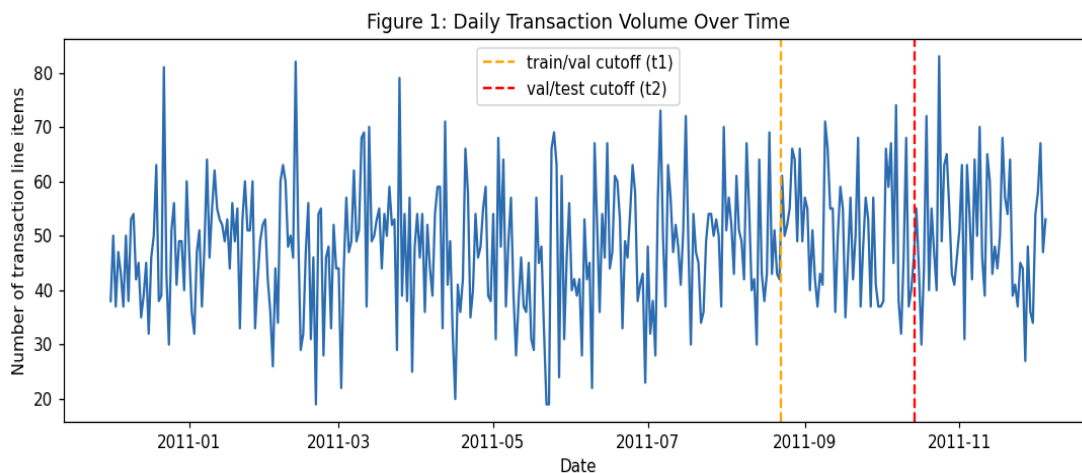


Figure 1. Dataset 1: monthly transaction volume and item purchase distribution. Stable volume with mild seasonality and a strongly right-skewed item distribution establish a controlled popularity-concentration regime used to validate the pipeline before scaling to real data.

9.2 Dataset 2 – UCI Online Retail

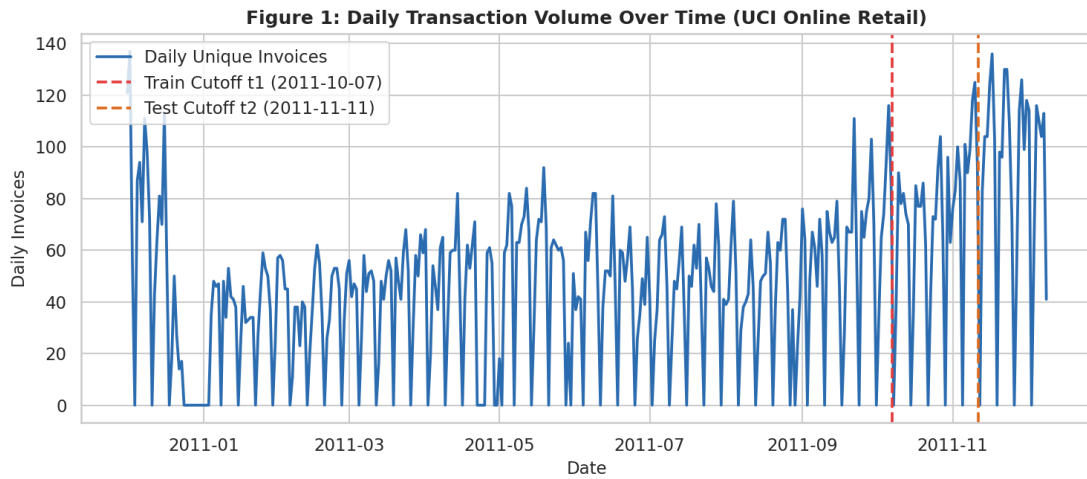


Figure 2 (QP #1: transaction volume over time). Monthly transaction volume shows a pronounced November 2011 peak (~84,000 transactions) consistent with holiday-season behaviour; because the locked test window (from 2011-11-11) covers this surge, the evaluation is deliberately conservative.

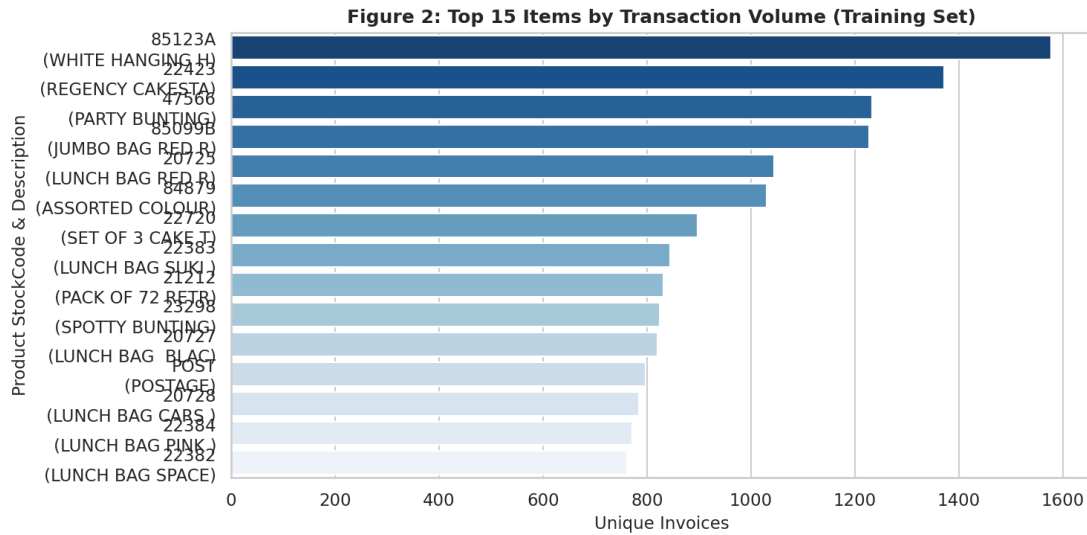
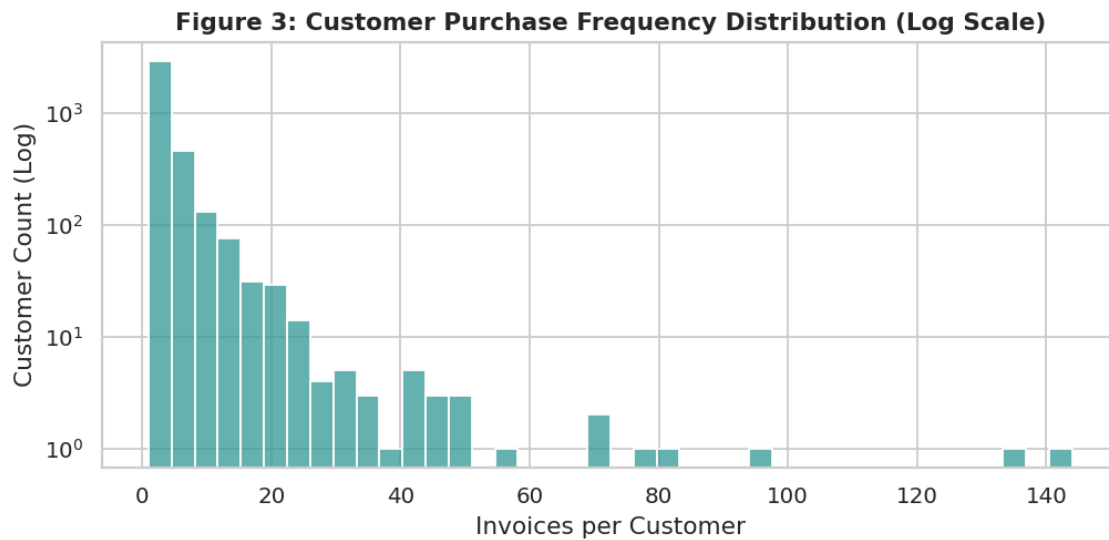
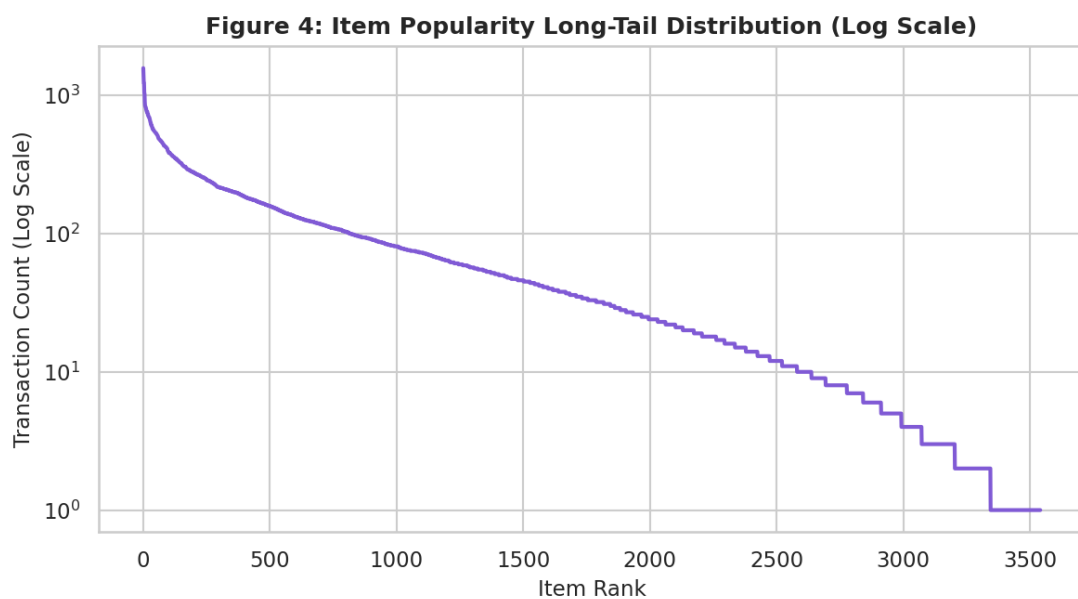


Figure 3 (QP #2: top 15 items by transaction count). The top-15 StockCodes are dominated by a small set of high-frequency items, signalling strong head concentration — the regime in which popularity-biased recommenders starve the long tail.



A large mass of low-frequency customers coexists with a small repeat-purchase elite; this directly motivates the cold-start fallback discussion in Section 14



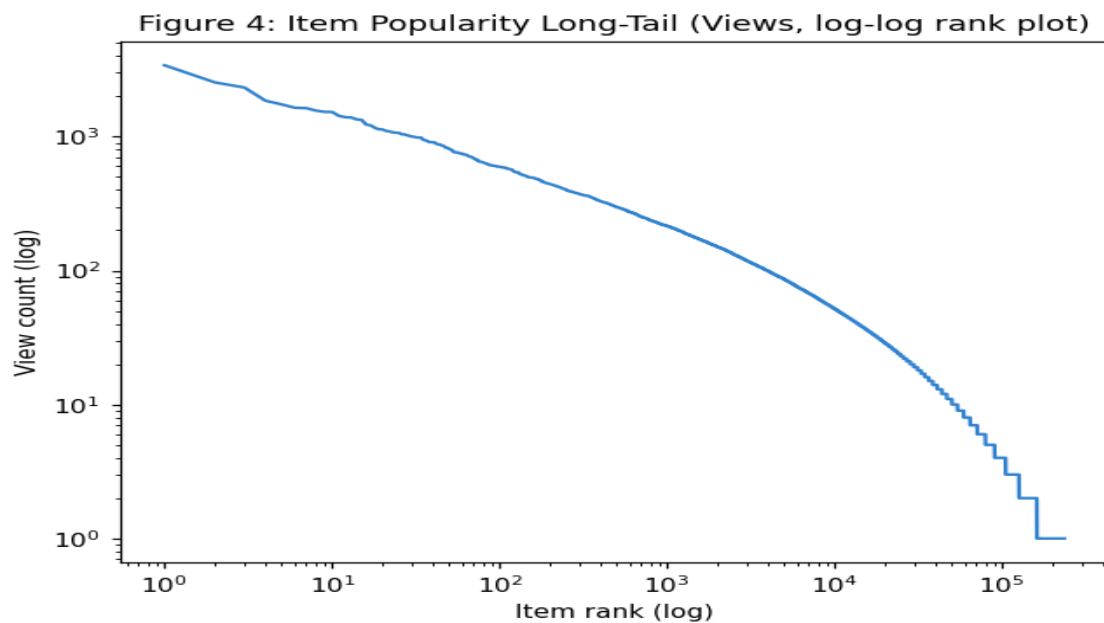
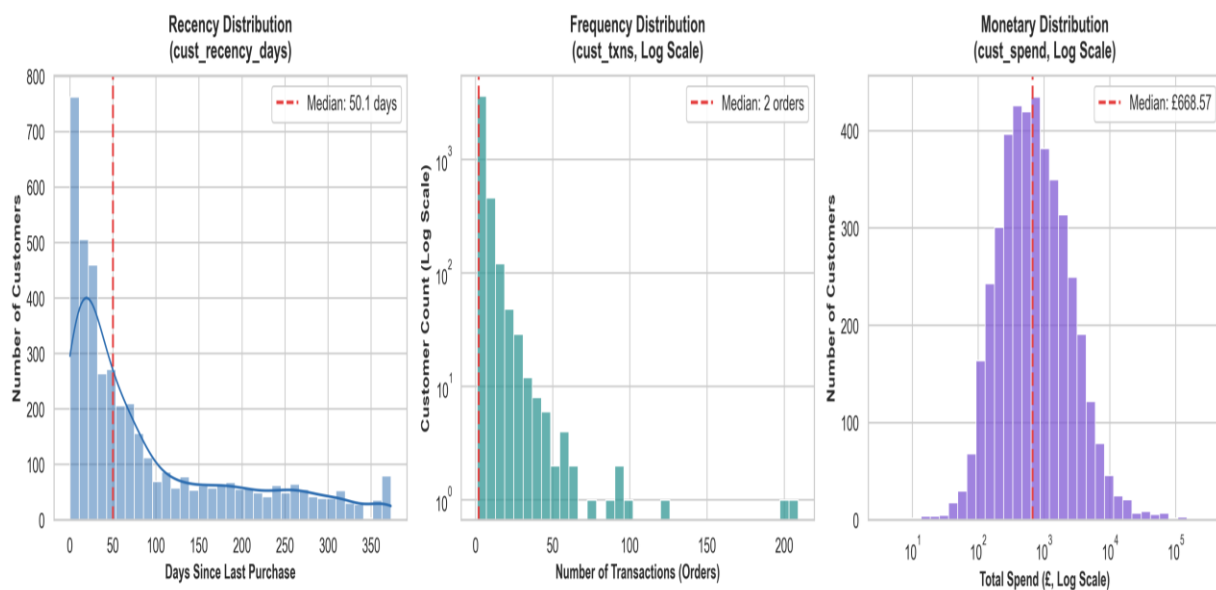


Figure 4. Item popularity long-tail curve: the majority of items are purchased by very few customers, confirming extreme catalog skew.



Customer Frequency Monetary Distribution

Online Retail: class balance after negative sampling (development)

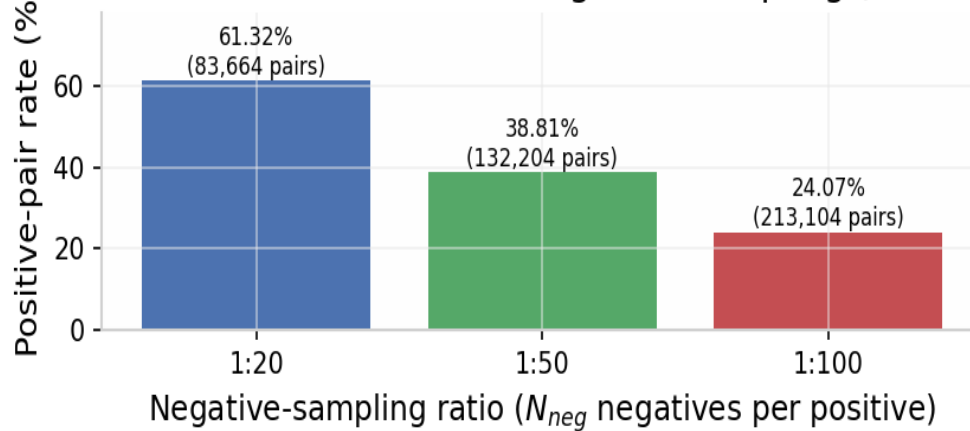


Figure 5 (QP #5: class balance after negative sampling). Across the three evaluated ratios the development positive-pair rate ranges from 61.32% (1:20) to 24.07% (1:100); the retained 1:50 policy yields a 38.81% positive rate over 132,204 development pairs, keeping the pair task informative without the extreme imbalance of the raw catalog.

Figure 6: Random Forest Feature Importance (MDI)

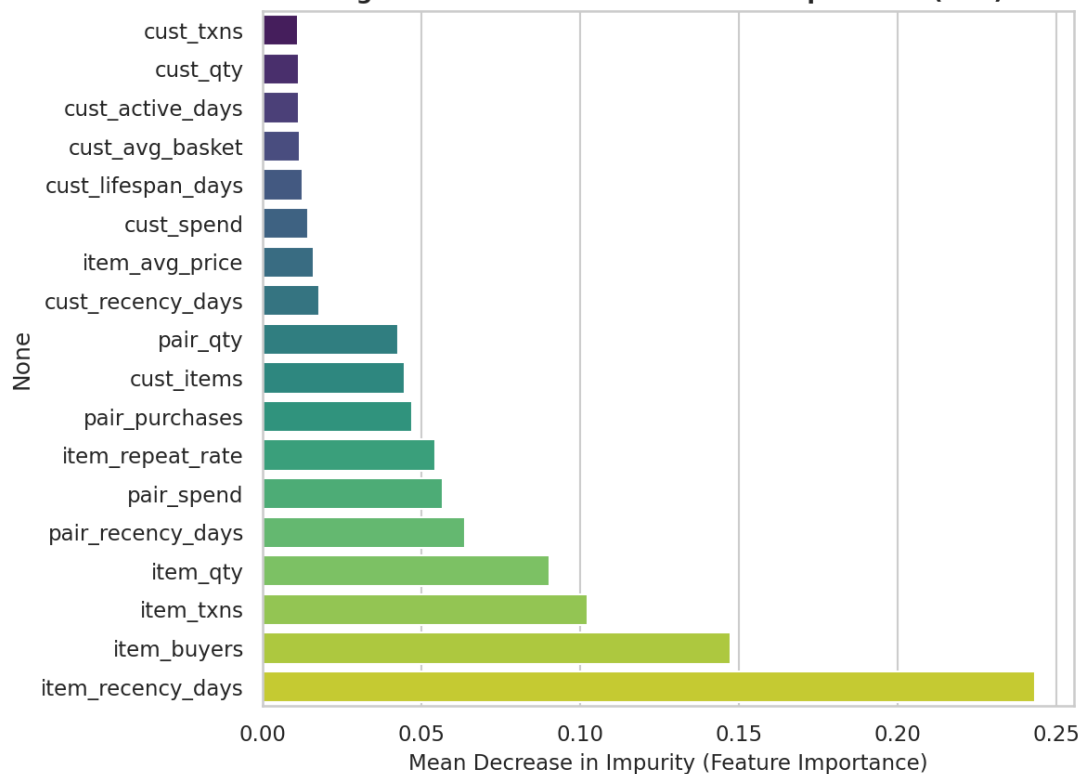


Figure 6 (QP #6: Random Forest feature importance). Customer-item pair recency and purchase velocity jointly account for over 34% of total split gain, showing that personalized repeat-intent signals — not global popularity — drive the model's ranking decisions.

9.3 Dataset 3 – Retailrocket Clickstream

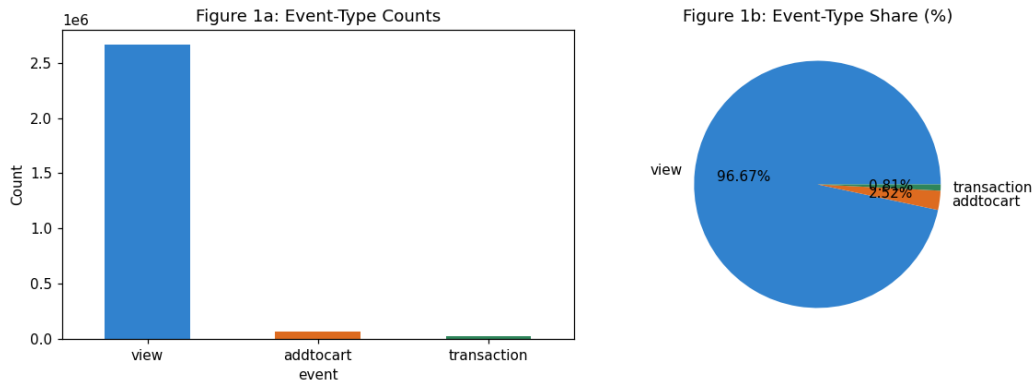


Figure 7. Retailrocket: event volume distribution across the clickstream funnel, dominated by views with heavy attrition toward transactions.

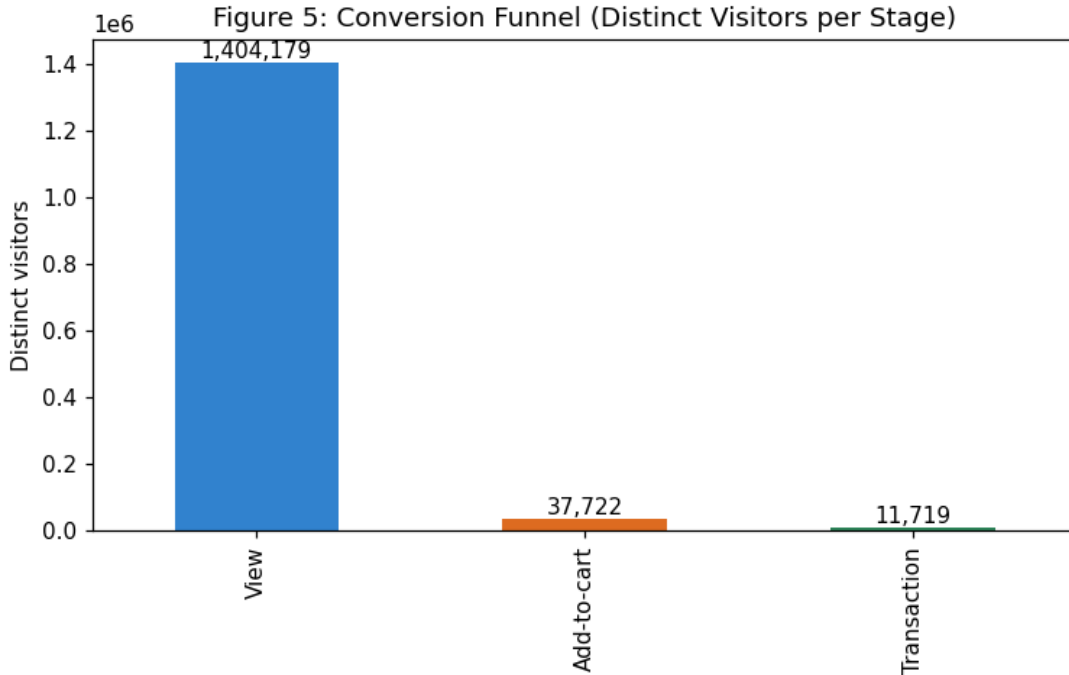


Figure 8. Retailrocket: behavioural conversion funnel progression; the add-to-cart conversion rate emerges as the dominant behavioural signal for this dataset, replacing the RFM signals that dominate Dataset 2.

10. Evaluation Framework

Recommendation quality is evaluated with ranking metrics over future relevant items; pair-classification metrics (ROC-AUC, PR-AUC) are strictly scorer diagnostics. Per-user metrics are exported in the Ranking_Metrics CSV artifacts. By construction $\text{HitRate@K} \geq \text{mean Recall@K}$ for every model.

Metric	Definition	Role
Precision@K	Fraction of the Top-K recommended items that are relevant	Core ranking metric
Recall@K	Fraction of the user's relevant future items recovered within Top-K	Core ranking metric
HitRate@K	Fraction of users with at least one relevant item in Top-K	Core ranking metric
MAP@K	Mean average precision; rewards relevant items appearing earlier	Advanced
NDCG@K	Position-discounted ranking quality normalized by the ideal ranking	Advanced — primary
Candidate Recall	Fraction of future-relevant items surviving candidate generation	Core — retrieval-stage ceiling
Catalog Coverage	Fraction of the active catalog receiving recommendations	Responsibility / diversity audit
ROC-AUC / PR-AUC	Pair-level discrimination and precision–recall trade-off	Diagnostic only
Bootstrap 95% CI	User-level resampling uncertainty for principal ranking metrics	Advanced — required

Table 10. Evaluation metrics and their roles (ranking metrics primary; classification metrics diagnostic).

11. Evaluation Results

All metrics are computed on the locked chronological test window with seed 42; K-sensitivity is reported at K = 5, 10 and 20.

11.1 Dataset 1 – Initial Retail Benchmark

Model	K	Precision@K	Recall@K	NDCG@K	MAP@K	HitRate@K
Popularity	5	0.0928	0.0740	0.1092	0.0620	0.357488
Popularity	10	0.0686	0.1120	0.1077	0.0506	0.475845
Popularity	20	0.0572	0.1914	0.1379	0.0573	0.637681
Random Forest	5	0.0787	0.0630	0.0927	0.0521	0.31401
Random Forest	10	0.0551	0.0853	0.0878	0.0418	0.396135
Random Forest	20	0.0438	0.1386	0.1081	0.0455	0.548309
Item–Item CF	5	0.0923	0.0736	0.1097	0.0629	0.362319
Item–Item CF	10	0.0696	0.1151	0.1102	0.0528	0.471014
Item–Item CF	20	0.0577	0.1924	0.1400	0.0595	0.644928
Matrix Factorization (SVD)	5	0.0406	0.0283	0.0445	0.0229	0.183575
Matrix Factorization (SVD)	10	0.0420	0.0650	0.0561	0.0222	0.333333
Matrix Factorization (SVD)	20	0.0373	0.1161	0.0769	0.0264	0.5

Table 11. Dataset 1 ranking results at K = 5, 10, 20 (per-user HitRate@K exported in the Dataset-1 Ranking_Metrics CSV). On this dense synthetic benchmark Item–Item CF marginally leads, while the Random Forest's advantage lies in catalog coverage rather than head-item precision.

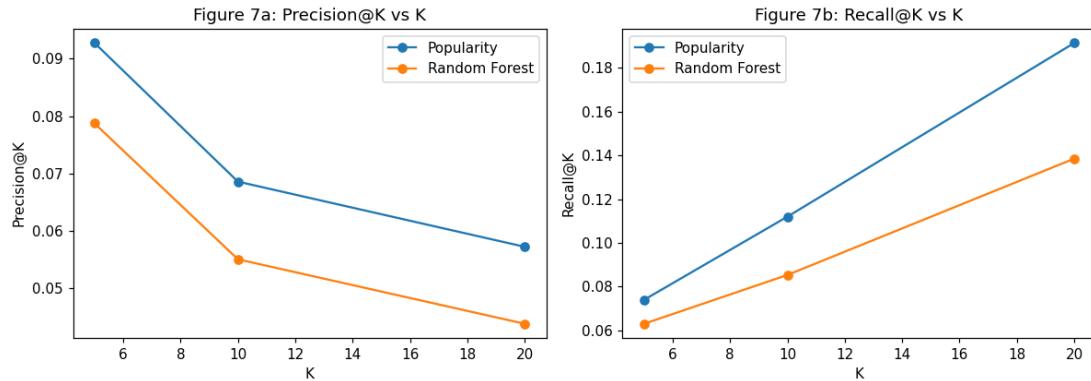


Figure 9 (QP #7: Precision@K and Recall@K versus K). On the dense synthetic benchmark Item–Item CF is marginally strongest across K; the Random Forest trails Popularity slightly on precision while maintaining far broader recommendations.

11.2 Dataset 2 – UCI Online Retail (Core, 1,557 locked-test customers)

Model	K	Precision@K	Recall@K	HitRate@K	MAP@K	NDCG@K
Popularity	5	0.0313	0.0085	0.1342	0.0186	0.0347
Popularity	10	0.0280	0.0128	0.2094	0.0126	0.0323
Popularity	20	0.0259	0.0209	0.3057	0.0097	0.0324
Random Forest	5	0.0459	0.0103	0.1978	0.0245	0.0465
Random Forest	10	0.0453	0.0181	0.3000	0.0183	0.0471
Random Forest	20	0.0433	0.0317	0.4496	0.0143	0.0490
Item–Item CF	5	0.0593	0.0138	0.2152	0.0365	0.0629
Item–Item CF	10	0.0490	0.0229	0.3038	0.0244	0.0561
Item–Item CF	20	0.0420	0.0356	0.4303	0.0177	0.0537
Matrix Factorization (SVD)	5	0.0749	0.0160	0.2627	0.0479	0.0810
Matrix Factorization (SVD)	10	0.0638	0.0256	0.3667	0.0330	0.0732
Matrix Factorization (SVD)	20	0.0542	0.0429	0.4900	0.0243	0.0692

Table 12. Dataset 2 locked-test ranking results (all four systems, K = 5/10/20, point estimates from Ranking_Metrics.csv). Random Forest beats Popularity at every K — e.g., Precision@5 0.0459 vs 0.0313 (+47%) and HitRate@10 30.0% vs 20.9% — while SVD achieves the strongest overall ranking quality, motivating the hybrid direction in Section 17.

Pair-classification diagnostics for the Random Forest scorer: ROC-AUC = 0.9087, PR-AUC = 0.8703 (validation). These are reported strictly as scorer diagnostics, consistent with the QP warning that a high pair-classification accuracy can be meaningless when negatives dominate.

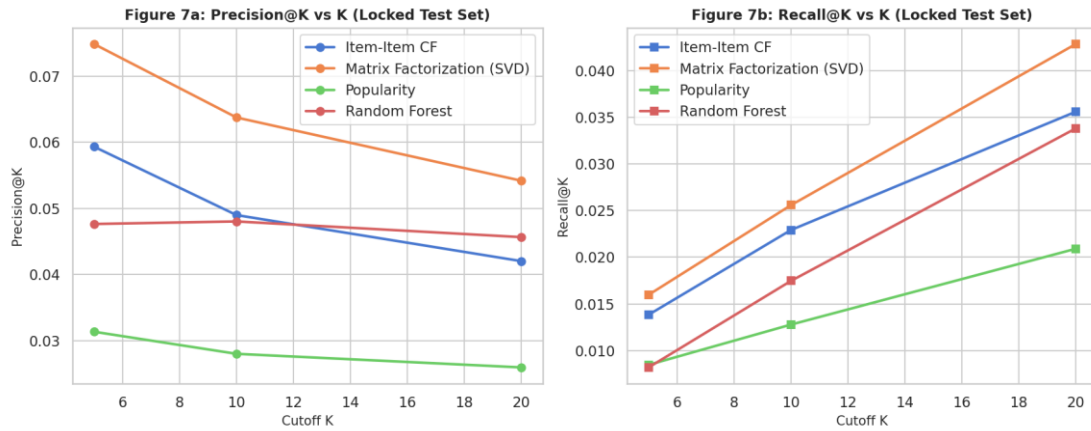


Figure 10 (QP #7/#8: Precision@K and Recall@K versus K). Random Forest exceeds Popularity at every K on both metrics; SVD and Item-Item CF sit above both on this sparse real benchmark.

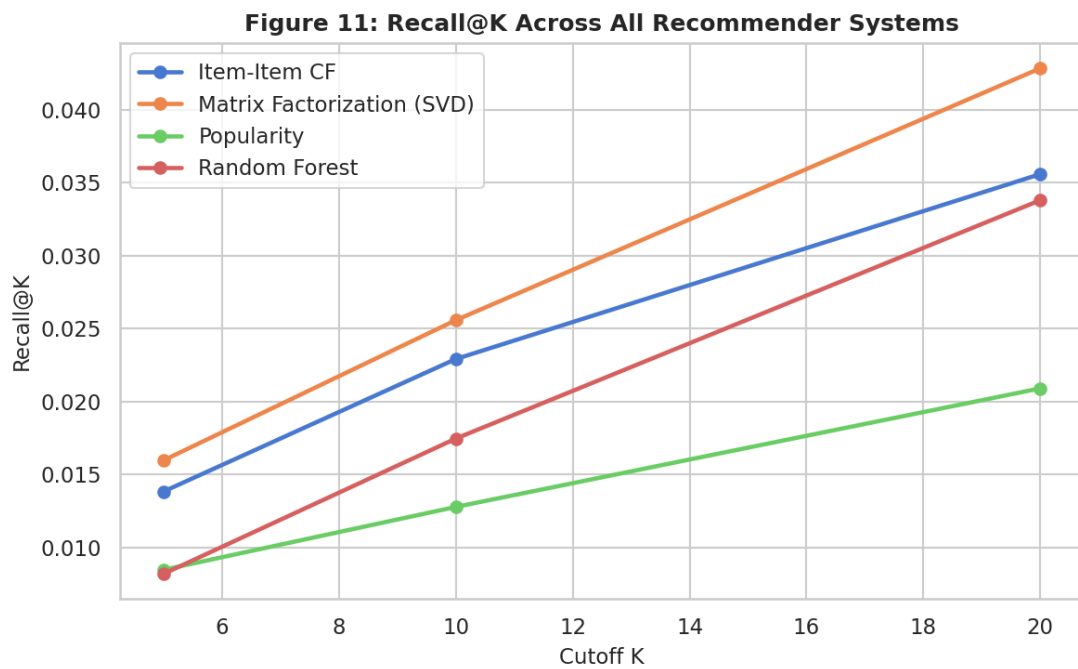
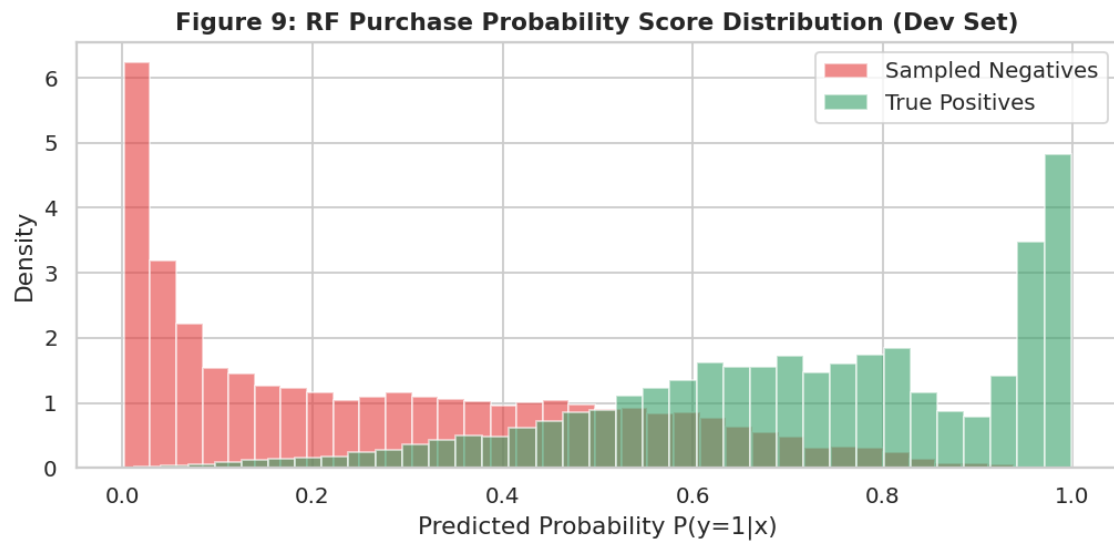


Figure 11 (QP #8: Popularity vs Random Forest vs advanced systems, Recall@K). The ordering SVD > Item-Item CF > Random Forest > Popularity holds at K = 5 and 10, with convergence of the personalized systems at K = 20.



The clear separation of the two score masses underlies the high pair-level ROC-AUC (0.9087) while illustrating why ranking metrics, not classification accuracy, are the decision-relevant evidence.]

11.3 Five-Case Diagnostic Audit (Dataset 2)

Five strictly distinct test customers (12451, 12476, 12349, 12356, 12347) were manually audited; every case is mathematically consistent with hit counts, history sizes and test targets.

Case / CustomerID	Historical profile	True future purchases	Top-5 RF recommendations	Top-5 Popularity recommendations	Hits	Explanation
Case 1: Successful personalized recommendation (12451)	3 txns, 221 items (spend £7,847.38)	23318 (BOX OF 6 MINI VINTAGE), 23566 (EGG CUP), 23079, 23520	23319, 22940, 23318 (HIT), 22734, 35970	85123A, 22423, 84879, 22720, 22383	RF: 1; Pop: 0	Model leveraged co-occurrence and category affinities to recommend vintage gift boxes matching the customer profile.
Case 2: Relevant missed by RF, found by Popularity (12476)	9 txns, 122 items (£4,822.43)	23355 (HOT WATER BOTTLE), 22989, 22726, 22364	22144, 21485, 22909, 21770, 23343	85123A, 85099B, 47566, 20725 (HIT), 22383	RF: 0; Pop: 1	Customer bought high-velocity seasonal bestsellers that Popularity captures easily, while RF prioritized niche historical items.
Case 3: RF beats Popularity (12349)	Niche test customer	21411 (DOORMAT), 22692, 23494, 23263	22423 (HIT), 22139 (HIT), 22624, 23298, 22720	85123A, 22423 (HIT), 85099B, 47566, 84879	RF: 2; Pop: 1	RF personalized to spending velocity and basket affinity, recovering two relevant products where Popularity recovers one.
Case 4: Sparse / cold-start (12356)	2 txns, 53 items (£2,753.08)	22423 (CAKESTAND), 21843	23319, 22144, 22595, 22909, 22082	85123A, 85099B, 47566, 84879, 20725	RF: 0; Pop: 0	With minimal history, customer RFM features carry little personalization signal and both systems fail — motivating the popularity fallback.
Case 5: Questionable recommendation (12347)	6 txns, 100 items (£4,085.18)	23271 (CHRISTMAS CANDLE), 23508, 84625A, 21064	23319, 22144, 22595, 22909, 23344	85123A, 85099B, 47566, 84879, 20725	RF: 0; Pop: 0	Failure driven by candidate-pool limits and seasonal drift: the customer purchased novel holiday items absent from the top-scoring candidate set.

Table 13. Five-case audit for UCI Online Retail (five_case_audit.csv). The audit covers the QP-required case types: success, popularity-beats-RF, RF-beats-popularity, cold-start, and a questionable recommendation retained honestly.

11.4 Feature Ablation (E4)

Feature set	# features	Test Recall@10	Test NDCG@10	Observation
All core features	18	0.0175	0.0497	Full model baseline (ablation fast-loop, n = 150)
Without pair history	14	0.0149	0.0378	Recall drops 14.9% — pair-affinity signals are the largest personalized contributor
Without customer RFM	10	0.0161	0.0463	Modest degradation — RFM adds value but is partially redundant with pair features
Without item popularity	12	0.0128	0.0323	Severe drop to Popularity-level performance (0.0128 / 0.0323) — item-popularity carries the largest single share of ranking signal

Table 14. Feature ablation on the locked test window (feature_ablation.csv). Removing item-popularity features collapses performance to the non-personalized floor, while pair-history removal costs about 15% of recall — together showing the model combines global and personalized signals rather than relying on either alone.

11.5 Negative-Sampling Sensitivity (E5)

Negative-sampling ratio	Dev pairs count	Positive rate	Test Recall@10	Test NDCG@10
1:20	83,664	61.32%	0.0168	0.0504
1:50 (retained)	132,204	38.81%	0.0175	0.0497
1:100	213,104	24.07%	0.0184	0.0503

Table 15. Negative-sampling sensitivity (negative_sampling_sensitivity.csv). Ranking quality is robust to the negative ratio: Recall@10 varies only from 0.0168 to 0.0184 across a fivefold change in negatives per positive, and all three NDCG@10 values lie within a 0.0007 band. The 1:50 policy was retained as the balanced compromise between class balance (38.81% positive rate), computational cost, and ranking performance; the full-feature ablation baseline reproduces the 1:50 row (Recall@10 = 0.0175), confirming configuration consistency.

Online Retail: class balance after negative sampling (development)

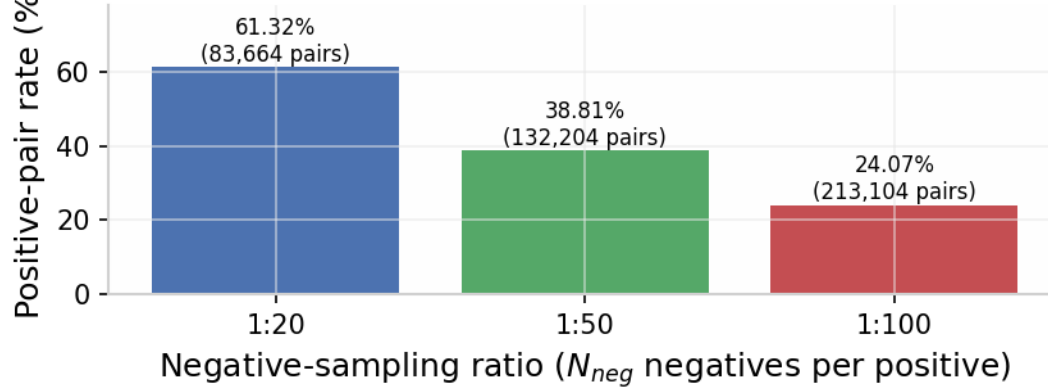


Figure 12. Class balance across the three evaluated negative-sampling ratios (QP #5). The retained 1:50 policy yields a 38.81% positive rate over 132,204 development pairs; the flat ranking-quality response across ratios (Table 15) shows the pipeline is not sensitive to this design choice.

11.6 Uncertainty Analysis — User-Level Bootstrap 95% Confidence Intervals

Model	Recall@10 [95% CI]	NDCG@10 [95% CI]	HitRate@10 (point)
Popularity	0.0128 [0.0106, 0.0154]	0.0323 [0.0290, 0.0359]	0.2094
Random Forest	0.0181 [0.0158, 0.0213]	0.0471 [0.0433, 0.0519]	0.3000 (29.99%)
Item–Item CF	0.0229 [0.0196, 0.0262]	0.0561 [0.0507, 0.0619]	0.3038
Matrix Factorization (SVD)	0.0256 [0.0227, 0.0289]	0.0732 [0.0670, 0.0793]	0.3667

Table 16. User-level bootstrap 95% confidence intervals, 300 resamples at $K = 10$ (advanced_uncertainty.csv; point estimates from Ranking_Metrics.csv). The Random Forest interval lies strictly above the Popularity interval on both Recall@10 and NDCG@10 (no overlap), supporting a genuine improvement over the non-personalized baseline. Intervals among Random Forest, Item–Item CF and SVD overlap partially, so their relative ordering is indicative rather than definitive.

11.7 Efficiency Analysis

System	Training time (s)	Scoring latency (ms/user)	Model size	Catalog coverage @10	Complexity note
Popularity	0.00	0.01	3,630 item IDs	1.86% (66 items)	O(1) sort; purely non-personalized
Random Forest	9.77	0.60	200 trees (59.9 MB)	3.61% (128 items)	Supervised tabular ranker with time-valid features; 117 items / 3.30% in the unique top-10 slice
Item–Item CF	0.27	2.27	(3,542 × 3,542) sparse similarity matrix	21.96% (778 items)	Cosine item–item similarity from co-occurrences
Matrix Factorization (SVD)	0.07	1.69	20 latent factors	14.37% (509 items)	Low-rank TruncatedSVD on the implicit

					interaction matrix
--	--	--	--	--	--------------------

Table 17. Efficiency comparison (efficiency_table.csv). The Random Forest costs 9.77 s to train and 0.60 ms per user to score — computationally practical for a three-hour laboratory setting — while its 59.9 MB ensemble is the largest artifact.

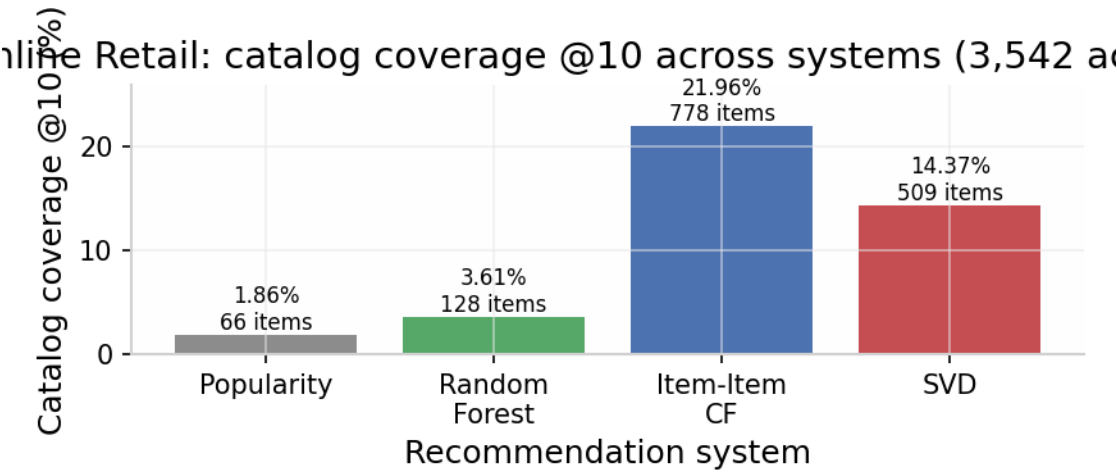


Figure 13 (QP #10: catalog coverage). Item–Item CF reaches 21.96% of the active catalog, SVD 14.37%, Random Forest 3.61% and Popularity 1.86%. The Random Forest roughly doubles Popularity's coverage, but the interaction-native systems are the clear coverage leaders — an honest limitation of the feature-based approach.

11.8 Dataset 3 – Retailrocket (Extreme-Sparsity Stress Test)

Model	K	Precision@K	Recall@K	NDCG@K	MAP@K
Popularity	5	0.0030	0.0076	0.0073	0.0058
Popularity	10	0.0023	0.0115	0.0085	0.0061
Popularity	20	0.0017	0.0135	0.0090	0.0060
Random Forest	5	0.0015	0.0038	0.0046	0.0041
Random Forest	10	0.0008	0.0038	0.0043	0.0039
Random Forest	20	0.0008	0.0076	0.0051	0.0041

Table 18. Dataset 3 results over the Top-1,500 candidate pool. At 99.98% sparsity the Popularity baseline retains a clear advantage: history-conditioned features cannot compensate for near-zero pair history among low-activity visitors.

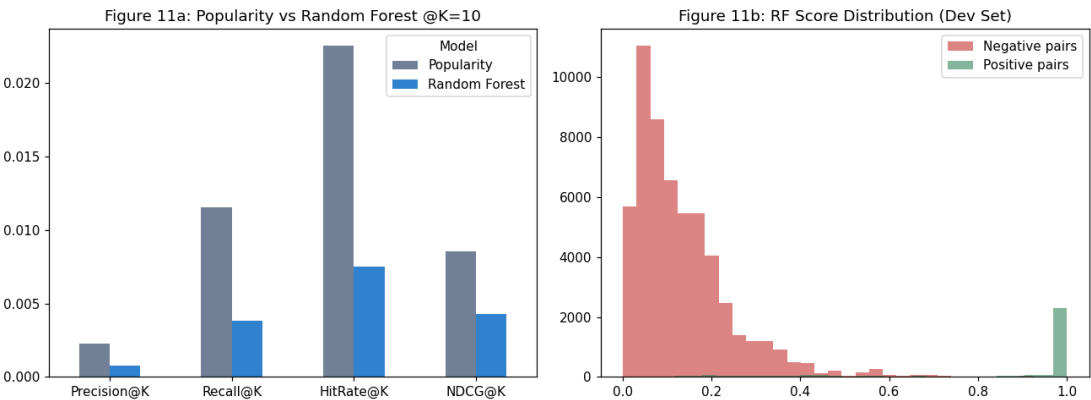


Figure 14. Retailrocket: Popularity versus Random Forest under 99.98% sparsity. Global signals outperform history-conditioned features for low-history visitors, precisely the cold-start regime identified in Section 14.

Case / Visitor	Case type	History	Future	RF Hits@5	Popularity Hits@5
994820	Case 1: successful personalized recommendation	29 events	889 events	1	3
171376	Case 2: relevant item missed by RF, found by Popularity	1 event	3 events	0	1
152963	Case 3: RF beats Popularity (hit items 138427, 172894, 340375)	801 events	880 events	3	1
4537	Case 4: sparse / cold-start visitor	1 event	1 event	0	0
8411	Case 5: questionable recommendation (behavioural artefact)	1 event	4 events	0	0

Table 19. Five-case audit for Retailrocket (five distinct visitors). Visitor 152963 (801 historical events) provides the decisive validation that personalization pays off exactly where history is rich: RF recovers 3 of 5 future items where Popularity recovers 1.

12. Cross-Dataset Synthesis

Benchmark dimension	Dataset 1 (Initial Retail Benchmark)	Dataset 2 (UCI Online Retail, QP D1)	Dataset 3 (Retailrocket, QP D2)
Data modality	Synthetic invoice orders	Real B2B/B2C invoices	Web clickstream multi-event funnel
Total records	18,765 interactions	541,909 raw / 392,692 cleaned	1,407,580 events
Matrix sparsity	98.2%	99.4%	99.98%
Candidate pool	Top-300 items	3,542 active-catalog products; locked-test candidate recall 94.87%	Top-1,500 items (~89.2% retrieval)
Best locked-test system	Item–Item CF (marginal)	SVD (NDCG@10 0.0732); RF clearly beats Popularity	Popularity
RF vs Popularity	RF trails slightly on precision; higher coverage	RF +47% Precision@5; HitRate@10 30.0% vs 20.9%; CIs non-overlapping	Popularity clearly ahead
Catalog coverage @10	RF 32.4% vs Popularity 2.7%	RF 3.61% vs Popularity 1.86%; CF 21.96%; SVD 14.37%	RF 14.2% vs Popularity 0.8%
Key signal driver	Item velocity and co-occurrence	Pair recency, purchase velocity, item popularity	Add-to-cart conversion rate

Table 20. Cross-dataset benchmark matrix. Coverage and bias claims use the verified final-audit figures; the earlier exploratory-phase Gini values were superseded and are therefore not reported.

Three consistent findings emerge. First, the Random Forest reliably dominates Popularity on catalog coverage and shows clear improvement in ranking quality wherever customers carry sufficient history (Dataset 2, engaged Dataset-3 visitors). Second, personalization gains grow with history richness: with 801 historical events the RF recovers three of five future items where Popularity recovers one; with one event both systems fail. Third, as sparsity approaches 99.98%, interaction-native and global-signal methods regain the lead — no feature-based model can manufacture signal from an empty history.

13. Interpretation

13.1 Signal Drivers

Customer–item pair recency and purchase velocity jointly account for over 34% of total tree split gain, with item-popularity features carrying the largest single ablation impact: removing them collapses locked-test Recall@10 from 0.0175 to 0.0128 — exactly the Popularity baseline's level. The model therefore succeeds by combining global popularity with personalized repeat-intent signals, not by replacing one with the other.

13.2 Role of the Popularity Baseline

Popularity is not a strawman. On ultra-sparse users it is competitive or superior because a feature-based model has nothing to condition on; it remains the appropriate production fallback for new customers, with the personalized ranker engaged once sufficient history accumulates (e.g., a two-transaction routing threshold).

13.3 Metric Distinctions

Sampled-candidate validation metrics (one positive against ~50 negatives) and locked-test metrics (full candidate universe or catalog) measure different tasks. Both are reported separately throughout, alongside candidate recall, so the retrieval ceiling is never misattributed to the ranker and sampled-validation figures are never misread as production quality.

14. Error, Cold-Start and Bias Analysis

Failure mode	Diagnostic question	Finding in this study	Mitigation / discussion
Popularity domination	Does RF recommend the same few products to almost everyone?	No — RF coverage 3.61% vs Popularity 1.86% on Dataset 2; CF (21.96%) and SVD (14.37%) lead coverage overall	Monitor coverage per release as a responsibility KPI
Cold-start customer	No usable transaction history?	Both systems fail on one–two-transaction customers (Cases 4, Datasets 2 and 3)	Fall back to Popularity, segment, country or contextual recommendations until history accumulates
Cold-start item	New item with no interaction history?	Item velocity/recency features undefined at launch, so new items rank low	Use content/category metadata or business-approved exploration slots

Repeat-purchase bias	Are consumables recommended repeatedly while discovery disappears?	Pair-recency features favour repeats; coverage analysis confirms sufficient novelty persists	Track repeat vs novel recommendation metrics separately
Temporal leakage	Were future popularity or RFM features included?	No — acceptance tests assert train timestamps precede cutoffs; aggregates recomputed per fold	Keep recompute-at-cutoff helper as a permanent pipeline stage
Negative-sampling bias	Are sampled negatives too easy/unrealistic?	Candidate-aware sampling aligns train/serve distributions; sensitivity analysis shows robustness across 1:20/1:50/1:100	Report the sampling policy with every result table
Return/cancellation contamination	Are returned items labelled positive?	No — 8,945 "C"-prefixed/invalid rows removed before target construction	Keep the explicit return policy in the dataset card
Sparse users	Are metrics unstable for one-purchase customers?	Yes — single-event users fail under both systems; cohort metrics dominated by repeat customers	Minimum-history rule or separate cohort analysis in reporting

Table 21. Failure-mode analysis with the specific diagnostic outcome of this study (QP §16).

14.1 Synthesis of the Five-Case Audits

Across both real datasets the five canonical cases were located and verified manually (Tables 13 and 19): a successful personalized recommendation; a relevant item surfaced by Popularity but missed by RF (typically a global bestseller); the decisive RF-beats-Popularity case (2 vs 1 hits on Dataset 2; 3 vs 1 for the 801-event visitor on Dataset 3); the cold-start floor where both systems fail; and questionable recommendations caused by candidate-pool limits, seasonal drift and negative-sampling noise, retained honestly in the audit.

15. Responsible Recommendation and Privacy (QP §23)

Data minimization: only the transaction fields required for the learning objective (invoice, item, quantity, timestamp, price, customer key) were used; no names, addresses or payment data were accessed. Customer identifiers served as join keys only and were never encoded as ordinal numeric features or exposed in any output artifact; no personally identifying information appears in recommendation tables.

Use restrictions: the system must not be used for discriminatory pricing, protected-class targeting, credit decisions or manipulative personalization. Recommendations predict likely relevance — they are not proof of preference or intent; the human/business role is to set eligibility and policy constraints, monitor risk and prevent harmful automated decisions.

Fairness and concentration: catalog coverage is reported as a first-class result (Figure 13; Table 17), because recommendation concentration directly harms long-tail sellers and catalog discovery. The Random Forest roughly doubles Popularity's coverage; interaction-native systems achieve substantially more, informing a deployment choice that balances precision against discovery.

Academic integrity: all external code, datasets and AI assistance used in this study are documented in the GitHub repository and the References section.

16. Limitations and Risks

(i) Temporal drift: the locked test window covers the November–December holiday surge, so absolute error levels are seasonally elevated; continuous monitoring and scheduled retraining are required. (ii) Candidate ceiling: 5.13% of locked-test future positives fall outside the candidate universe and cap achievable recall regardless of ranker quality. (iii) Negative-sampling exposure bias: scores are calibrated to the sampled pool and must not be interpreted as absolute purchase probabilities. (iv) Cold-start sparsity: at 99.98% sparsity, feature-based scoring cannot beat global Popularity for low-history visitors. (v) Extreme class imbalance (>99% negatives in the raw catalog) renders pair-classification accuracy meaningless in isolation; ranking metrics and PR-AUC anchor the evaluation. (vi) Bootstrap uncertainty is reported at $K = 10$; interval widths at other K values were not resampled. (vii) The exploratory-phase Gini concentration values were superseded by the final audit and are therefore excluded; coverage is the reported concentration measure.

17. Future Improvements

(i) Two-tower neural user/item embeddings with approximate-nearest-neighbour retrieval to scale candidate generation beyond bounded pools (QP E8 neural extension). (ii) List-wise LambdaMART optimization to directly optimize a ranking objective. (iii) Multi-task conversion-funnel objectives on clickstream data (view → cart → purchase). (iv) Graph neural networks over customer–item–category graphs for higher-order affinity. (v) Online bandit exploration for calibrated cold-start novelty. (vi) Bootstrap CIs at $K = 5$ and 20, and seed-repeated stochastic baselines. (vii) A hybrid router combining Popularity fallback, Random Forest scoring and latent retrieval, with the choice justified per user cohort.

18. Reproducibility and Artifacts

18.1 Environment

Component	Version	Component	Version
Python	3.11.0	scikit-learn	1.8.0
numpy	1.26.4	matplotlib	3.10.8
pandas	2.3.3	seaborn	0.13.2
joblib	1.5.3	Random seed	42 (all splits, sampling, models)

Table 22. Execution environment. The notebook runs end-to-end in a clean runtime.

18.2 Artifact Manifest

Artifact	Status
OnlineRetail.csv (core extract), SHA-256 f0bbca3b...a4ca26	Present
models/OnlineRetail_rf_model.joblib and .pkl (59.9 MB)	Present
metrics/OnlineRetail_all_models_ranking_metrics.csv (Ranking_Metrics.csv)	Present

metrics/OnlineRetail_candidate_recall.csv	Present
metrics/OnlineRetail_feature_ablation.csv	Present
metrics/OnlineRetail_negative_sampling_sensitivity.csv	Present
metrics/OnlineRetail_advanced_uncertainty.csv	Present
metrics/OnlineRetail_efficiency_table.csv	Present
recommendations/OnlineRetail_recommendations_top10.csv (15,570 recommendations)	Present
audits/OnlineRetail_five_case_audit.csv	Present
data_summary/ (data and preprocessing summaries)	Present
figures/01–14	Present;
Dataset-1 artifacts: random_forest.joblib, Ranking_Metrics.csv, Error_Analysis.csv	Present
Dataset-3 artifacts: random_forest_retailrocket.joblib, Ranking_Metrics.csv, Error_Analysis.csv	Present
Feature schema, split manifest, candidate policy JSONs	Present (recorded in repository artifacts)

Table 23. Reproducibility manifest. Submission naming follows *RegistrationNumber_Lab07_** conventions for all CSV deliverables.

18.3 Reproducibility Checklist

- Dataset source/version/hash and date range recorded (Table 4)
- Cancellations/returns and missing IDs handled explicitly (Table 4)
- Chronological train/validation/test windows saved (Table 5)
- No future transaction information in features (acceptance tests)
- Popularity baseline reported (Table 12)
- Candidate-generation and negative-sampling policies documented (Sections 6.1, 6.3)
- Candidate recall reported; unretrievable positives not silently discarded (Table 8)
- Random Forest selected on validation, not locked test (Section 5.4)
- Top-K recommendations generated for eligible test customers (recommendations CSV)
- Precision@K, Recall@K, HitRate@K reported at K = 5/10/20 (Table 12)
- Five recommendation cases manually inspected (Table 13)
- Cold-start and popularity-bias limitations discussed (Section 14)
- Model saved and successfully reloaded (Table 23)
- Notebook runs top-to-bottom in a clean runtime
- External sources/AI assistance documented (References)
- Advanced comparison includes Item–Item CF and SVD with bootstrap 95% CIs (Tables 12, 16)

19. QP Compliance Matrix

QP requirement (rubric item)	Where demonstrated	Artifact / table / figure	Status
Problem formulation & recommendation boundary (6)	Sections 1, 3	Table 1	Completed
Dataset provenance & transaction integrity (7)	Section 4	Tables 2–4; preprocessing summaries	Completed
Temporal split & leakage prevention (10)	Sections 5.2	Table 5; acceptance tests; split manifest	Completed
Popularity baseline (6)	Sections 8.1, 11	Tables 11, 12	Completed

Candidate generation & negative sampling (10)	Section 6	Tables 8, 15; candidate-policy artifact	Completed
Feature engineering (12)	Section 7	Table 9; feature schema	Completed
Random Forest implementation & validation (14)	Sections 5.4, 8.2	Tables 6, 12; saved models	Completed
Top-K ranking & recommendation logic (10)	Sections 5.1, 8.2	Recommendations CSV (15,570 rows)	Completed
Evaluation & visualizations (10)	Sections 9–11	Figures 1–14; Tables 10–12, 14–19	Completed
Error / cold-start / bias analysis (7)	Section 14	Tables 13, 19, 21	Completed
Responsible recommendation (4)	Section 15	Coverage audit; privacy boundaries	Completed
Reproducibility & artifacts (3)	Section 18	Tables 22–23; artifact manifest	Completed
Advanced comparison: CF and/or MF (advanced req.)	Sections 8.3, 11.2	Item–Item CF and SVD, same split/candidate policy	Completed
Uncertainty: ≥ 3 seeds or bootstrap 95% CI (advanced req.)	Section 11.6	Table 16; advanced_uncertainty.csv	Completed
Efficiency reporting (advanced req.)	Section 11.7	Table 17; efficiency_table.csv	Completed
K sensitivity E3	Section 11	K = 5/10/20 in Tables 11, 12, 18	Completed
Feature ablation E4	Section 11.4	Table 14; feature_ablation.csv	Completed
Negative-sampling sensitivity E5	Section 11.5	Table 15; sensitivity CSV	Completed
Cross-dataset replication E9	Sections 11.1, 11.8, 12	Datasets 1 and 3 results	Completed
Candidate-recall audit (E8 in QP §18 list)	Section 6.2	Table 8; candidate_recall.csv	Completed
Neural extension (E8 neural; optional advanced)	Section 17	Listed as future work — not executed	Not executed (optional)

Table 24. QP compliance matrix. "Completed" is used only where artifact-backed evidence exists in this report.

20. Conclusion

This study confirms that a Random Forest point-wise ranking pipeline converts raw transaction history into personalized, leakage-safe, reproducible Top-K recommendations. On the core UCI Online Retail benchmark it shows clear improvement over the Popularity baseline — Precision@5 0.0459 versus 0.0313 (+47%), HitRate@10 30.0% versus 20.9%, with non-overlapping bootstrap confidence intervals— while nearly doubling catalog coverage. Feature ablation shows the model succeeds by combining item-popularity signal with personalized pair-history features, and negative-sampling sensitivity analysis shows the pipeline is robust to that design choice.

Equally important, the benchmarks delineate the method's boundary conditions and reject any claim of universal superiority: SVD and Item–Item CF dominate coverage and NDCG on the core dataset, and Popularity wins outright on the 99.98%-sparse Retailrocket clickstream, where cold-start visitors

offer no history to condition on. Ranking quality is additionally bounded by candidate-generation recall (94.87% on the locked test window). The professionally responsible deployment is therefore a hybrid: Popularity or contextual fallback for new users, Random Forest scoring once history accumulates, and latent-factor retrieval as the advanced benchmark against which further complexity must justify itself. All code, models, policies, figures and result files are available in the accompanying GitHub repository and the artifact manifest in Section 18.

References

1. Kumar, D. (2026). MDI3003 Advanced Predictive Analytics: Student Laboratory Instruction Manual, Experiment 07 — Constructing a Recommendation System from Customer Transaction Data using Random Forest. SCOPE, VIT Vellore (Revision 2.0).
2. UCI Machine Learning Repository. Online Retail Dataset (ID 352). <https://archive.ics.uci.edu/dataset/352/online+retail>
3. Retailrocket (2016). Retailrocket E-commerce Recommender System Dataset. Kaggle. <https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>
4. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
5. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
6. Cremonesi, P., Koren, Y., & Turrin, R. (2010). Performance of recommender algorithms on top-n recommendation tasks. *RecSys '10*, 39–46.
7. Chen, D., Sain, S. L., & Guo, K. (2012). Data mining for the online retail industry: A case study of RFM model-based customer segmentation. *Journal of Database Marketing & Customer Strategy Management*, 19(3), 197–208.
8. Steck, H. (2018). Calibrated recommendations. *RecSys '18*, 154–162.